CONSTRUÇÃO DE COMPILADORES · ESCOLA POLITÉCNICA · PUCRS

Atividade em grupo — Verificador Semântico da Mini Simplificada

Unidades 4.3 e 4.4 · código do verificador e experimentos · grupos de até 4 alunos

Nº	Nome	Matrícula
1		
2		
3		
4		

Orientações

- Grupos de até 4 alunos.
- Materiais: os slides "Tabela de Símbolos e Verificação Semântica" e o projeto mini-global-byaccj — versão simplificada da linguagem Mini, com apenas variáveis globais (incluindo arranjos de várias dimensões) e a função void main(). Leia o README.md e use o Makefile: make, make test, make tabela, make trace, make conflitos.
- O código mantém estruturas que esta versão ainda não usa (escopos, parâmetros, chamadas de função), reservadas para futuras extensões. Algumas questões tratam justamente delas.
- Responda em um documento separado, identificando cada resposta pelo código da questão (A1, A2, …). Sempre que possível, cite o arquivo e o trecho de código que fundamenta a resposta.
- Quando a questão pedir para executar um programa, cole a saída obtida e explique-a. Uma saída sem explicação não é considerada resposta.
- Todos os integrantes devem ser capazes de explicar qualquer resposta do grupo.

Parte C — O código do verificador

Projeto mini-global-byaccj: mini.flex, mini.y, Tipo.java, Simbolo.java, TabSimb.java, DeclId.java e Makefile.

C1 Descreva o caminho dos arquivos-fonte até o programa executável: que ferramenta processa cada arquivo, que arquivos são gerados e quem chama quem durante a execução. O que significam as opções -J e -Jnorun do byaccj e por que o main() Java está dentro de mini.y?

Referência: Makefile; README.md

C2 O que declaram %token <ival> NUM_INT e %type <obj> expr dims_decl indices? O que o léxico coloca em yylval ao reconhecer um ID e um NUM_INT? Por que as ações escrevem (Tipo)$1 e (List<Tipo>)$2? Na produção dims_decl : dims_decl '[' NUM_INT ']', que valor tem $3 e de que campo de ParserVal ele vem?

Referência: mini.y (%token, %type, dims_decl, indices); mini.flex

C3 Explique a estrutura de um programa imposta pelas produções programa, globais, decl_main e finais. Execute os dois programas abaixo e explique por que um gera erro sintático e o outro erro semântico.

Qual dos dois tipos de mensagem é mais útil para o programador? Como a gramática poderia ser alterada para que o programa (a) recebesse uma mensagem semântica clara?

```
// (a)
int a;
void main() { a = 1; }
int b;

// (b)
int a;
void f() { a = 1; }
```

Referência: mini.y — programa, globais, decl_main, finais, itens

C4 Explique a diferença entre busca(), buscaLocal() e resolve() e onde cada uma é usada em mini.y. Nesta versão só existem variáveis globais: quais escopos são abertos durante a análise de ok.mini e quantos podem estar abertos ao mesmo tempo? Por que, ainda assim, a pilha de escopos foi mantida?

Referência: TabSimb.java; mini.y — abreFuncao(), bloco

C5 Em que situação declara() emite erro e em que situação emite aviso de ocultação? É possível obter um aviso de ocultação nesta versão da linguagem? Justifique com o código.

Referência: mini.y — declara()

C6 Para a declaração "int a, t[2][5], b;", mostre o valor do atributo de dims_decl e de lista_ids após cada redução e diga em que momento os três símbolos são inseridos na tabela. Esses atributos são herdados ou sintetizados?

Referência: mini.y — item_id, dims_decl, decl_var; DeclId.java

C7 Desenhe a árvore da expressão de tipo construída para "double m[3][4]" e para "int c[2][3][4]". Explique por que o laço de declaraVars() percorre as dimensões da última para a primeira. Quais são os valores de dimensoes(), base() e toString() para c, c.elem e c.elem.elem?

Referência: Tipo.java (array, dimensoes, base, toString); mini.y — declaraVars()

C8 Acompanhe indexa() para m[i][j], m[1] e m[1][2][0] (com double m[3][4]) e para v[d] (com int v[5] e double d), indicando o tipo devolvido ou a mensagem. Em seguida, explique as mensagens das linhas 14 a 19 de exemplos/erros.mini. Por que a indexação parcial m[1] é aceita em uma expressão, mas não como alvo de atribuição?

Referência: mini.y — indexa(), atribui(); erros.mini linhas 14–19

C9 Calcule o resultado de igual() para os pares (int[3][4], int[5][4]), (int[3][4], int[3][5]) e (int[3], int[3][4]). Que regra de equivalência isso representa? Em que métodos igual() é usado nesta versão e por que, na prática, ele quase nunca é chamado com arranjos?

Referência: Tipo.igual(), Tipo.atribuivelA()

C10 Para a condição "m[2][3] > 10.5 && !achou" de ok.mini, liste em ordem os métodos de verificação chamados durante o parsing e o tipo devolvido por cada um, até checaCond().

Referência: mini.y — regras de expr; ok.mini

C11 No trace de tabela.mini, a linha 11 ("m[1][n] = v[n] * 1.5;") produz resolve n, resolve n, resolve v e só então resolve m. Explique por que m, que aparece primeiro no texto, é resolvido por último.

Referência: make tabela; exemplos/tabela.mini linha 11

C12 Por que laco é um contador e não um boolean? Como checaReturn() sabe que "return 1;" é inválido em main? Que parte desse código só fará diferença quando a linguagem tiver outras funções?

Referência: mini.y — cmd (WHILE, BREAK, RETURN), checaReturn(), funcAtual

C13 A produção bloco abre um escopo novo, mas nesta versão nada pode ser declarado dentro dele. Isso é um erro de projeto? Compare bloco com corpo e diga o que acontece na tabela, segundo o trace, ao entrar e sair de um bloco.

Referência: mini.y — bloco, corpo

C14 Identifique no código os métodos, campos e valores de enumeração que existem mas não são usados (ou nunca recebem dados) nesta versão. Para cada um, explique que papel teria em uma versão com funções, parâmetros e variáveis locais.

Referência: mini.y — seção "RESERVADO PARA FUTURAS IMPLEMENTAÇÕES"; Simbolo.java

Parte D — Experimentos

Execute, cole a saída relevante e explique. A explicação é o que será avaliado.

D1 Execute make tabela e explique cada uma das três impressões produzidas por $MOSTRA_TS: por que a primeira não contém fim nem main, por que a segunda mostra três escopos (dois vazios) e o que a terceira representa. Relacione com as linhas "abre escopo" e "fecha escopo" do trace.

Referência: make tabela; exemplos/tabela.mini

D2 Antes de executar, preveja o tipo de cada expressão ou a mensagem de erro do programa abaixo. Depois execute e compare. Explique qualquer diferença entre a previsão e o resultado.

```
int c[2][3][4];
int x;
double d;
void main() {
    x = c[1][2][3];
    d = c[1][2][3] / 2;
    x = c[1][2];
    x = c[1][2][3][0];
    c[0][0][0] = d;
    x = c[x][x][x] + c[0][0];
}
```

Referência: Tipo.java; mini.y — indexa(), aritmetico(), atribui()

D3 Permita a declaração de variáveis locais no início de main e dentro de blocos, alterando apenas a gramática. Mostre a alteração, execute make conflitos e teste com o programa abaixo, explicando cada aviso e a tabela mostrada pelo $MOSTRA_TS.

Que estruturas mantidas "para futuras implementações" tornaram essa mudança tão pequena?

```
int a, m[2][2];
void main() {
    int a;
    a = 1;
    { double a; int k; a = 2.5; k = m[1][0]; $MOSTRA_TS }
    int m;
}
```

Referência: mini.y — itens, declaraVars(), declara()

D4 Desafio (opcional): faça o verificador detectar índices constantes fora dos limites, como v[7] com int v[5] ou m[1][4] com double m[3][4]. Hoje o atributo de expr é apenas um Tipo. Que

informação precisaria ser acrescentada? Descreva (e, se possível, implemente) a solução e diga por que ela não resolve v[i] com i variável.

Referência: mini.y — expr, indexa(); slide "Responsabilidades da tabela de símbolos" (dobramento de constantes)